Diagnóstico del problema.

En el caso de PixelPerfect Studio, el principal problema es que el CSS fue creciendo sin un orden, lo que al ir implementando más proyectos a la ecuación, generó que cada programador fuera implementando clases sin un orden específico y esto provoca varios problemas.

Uno de los problemas más graves es que los estilos se pisan, ya que al usar clases genéricas (.titulo por ejemplo para una card o un footer), los estilos se mezclan y se rompe el programa. Otro problema es que hay reglas repetidas: el mismo color escrito en muchos lugares, lo que genera confusión y más trabajo al buscar cambiar todos.

Otro problema es la falta de jerarquía en las clases y no se entiende la estructura del HTML, impidiendo que en un equipo de trabajo más de una persona se encargue de solucionarlo sin que parezcan distintos entre sí. Por esta misma razón, crear componentes nuevos es un desafío por evitar modificar los estilos anteriores y por la desorganización respecto a donde ubicarlos.

Comparación y elección de metodología.

BEM	OOCSS	SMACSS
Las clases se nombran como bloque (.card), elemento (.card_titulo) y modificador (.card-destacada). No se repiten y son claramente identificables.	Separa la estructura de la apariencia para reutilizar estilos. Es más compleja de entender, pero sirve para no repetir el código.	Ordena el CSS en categorías. Ayuda a ordenar los archivos.

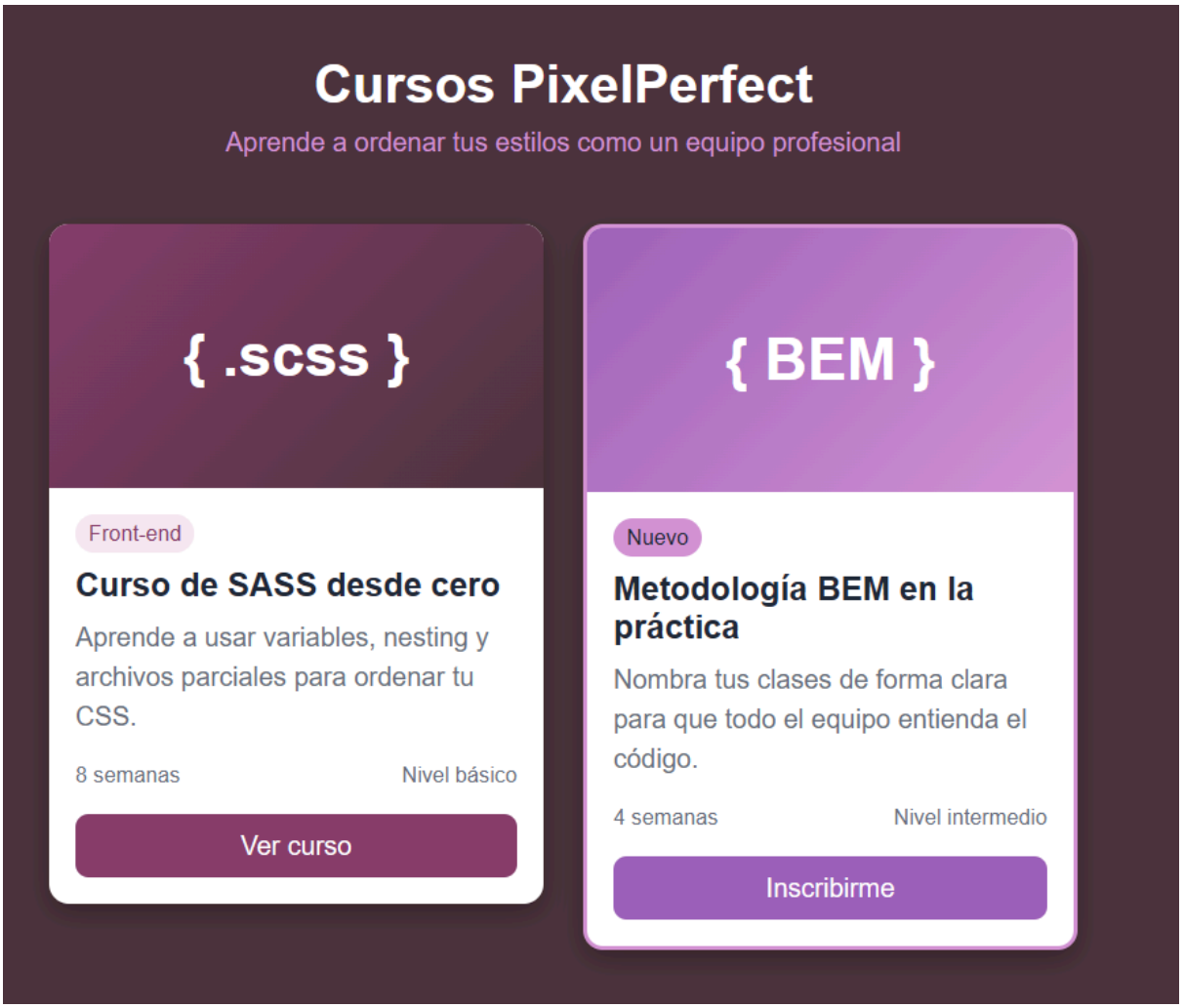
Yo elegí BEM porque el problema de la agencia es justamente que las clases chocan y no se sabe de dónde son. Con BEM cada clase lleva el nombre de su bloque, entonces .card__titulo nunca va a chocar con otro título. También creo que es la más fácil para que un equipo grande trabaje igual, porque son pocas reglas. Y se lleva bien con SASS porque con el & no hay que repetir el nombre del bloque. Para ordenar las carpetas igual usé una idea parecida a SMACSS (separar base y componentes).

Con BEM el código es más predecible (cada clase afecta solo a su componente), escalable (se agregan componentes nuevos en su propio archivo) y profesional (todo el equipo sigue la misma regla), que es justo lo que busca la agencia.

3. Implementación

Hice una página con un encabezado y dos cards de cursos. Una card es normal y la otra tiene el modificador card--destacada, que le pone un borde lila y cambia el color de la etiqueta. El botón de esa card usa el modificador card__boton--secundario para tener otro color. Así se ve cómo un modificador cambia un componente sin tener que crear uno nuevo.

Para los colores usé una paleta morada: fondo #4d353e, color principal #873e6d, secundario #9f63ba y acento #d495d6.



Las clases que usé son:

Clase	Tipo	Qué es
.card	Bloque	La tarjeta completa
.card__imagen	Elemento	Imagen del curso
.card__contenido	Elemento	Parte del texto

.card__etiqueta	Elemento	Categoría del curso
.card__titulo	Elemento	Nombre del curso
.card__descripcion	Elemento	Descripción
.card__info y .card__dato	Elemento	Duración y nivel
.card__boton	Elemento	Botón
.card--destacada	Modificador	Card con borde lila
.card__boton--secundario	Modificador	Botón de otro color
.encabezado, .encabezado__titulo, .encabezado__texto	Bloque y elementos	Título de la página

En SASS usé variables para los colores, la fuente, los tamaños y los espacios (por ejemplo $color-principal), así si quiero cambiar un color lo cambio en un solo lugar. También usé nesting con &, que es lo que hace que &__titulo se convierta en .card__titulo al compilar. Un pedazo del código:

```
.card {
  background-color: $color-blanco;
  border-radius: $borde-redondeado;

  &__titulo {
    font-size: $tamano-titulo;
  }

  &--destacada {
    border: 3px solid $color-acento;
  }
}
```

Para compilar usé la extensión Live Sass Compiler de VS Code, configurada para que guarde el styles.css al lado del index.html.

4. Documentación de la estructura

Separé el SCSS en varios archivos parciales (los que empiezan con _) y los junto todos en styles.scss. Así quedó la estructura:

```
proyecto/
|-- index.html
|-- styles.css
`-- scss/
    |-- styles.scss
    |-- abstracts/
    |   `-- _variables.scss
    |-- base/
    |   |-- _reset.scss
    |   `-- _base.scss
    `-- components/
        `-- _card.scss
```

styles.scss: no tiene estilos, solo importa los otros archivos. Es el que se compila.

abstracts/_variables.scss: aquí están los colores, fuentes, tamaños y espacios.

base/_reset.scss: quita los márgenes y estilos que pone el navegador por defecto.

base/_base.scss: estilos generales, el body, el encabezado y el contenedor de las cards.

components/_card.scss: todos los estilos de la card con BEM. Si después hago otro componente, por ejemplo un perfil, le creo su propio archivo en esta carpeta.

styles.css: es el CSS que sale al compilar y el que se conecta al HTML. Este no se edita a mano.

Lo ordené así porque primero tienen que ir las variables (si no, los otros archivos no las reconocen), después lo general y al final los componentes. Además así es fácil saber dónde buscar cada cosa.

Como cada proyecto de la agencia tiene su propio diseño, basta con cambiar el archivo _variables.scss para adaptar los mismos componentes a otro proyecto. Por ejemplo, para este trabajo cambié los colores originales por una paleta morada y solo tuve que tocar ese archivo.

5. Reflexión

Lo que más me costó fue entender bien qué era un bloque y qué era un elemento, y acordarme de cuándo iban los dos guiones bajos y cuándo los dos guiones. También al principio anidé de más en SASS, poniendo una clase dentro de otra dentro de otra, y después me di cuenta de que con BEM no hace falta porque cada clase ya tiene su nombre propio. Otra cosa fue que cambiaba los colores en el SCSS y la página seguía igual, porque el CSS compilado se estaba guardando en otra carpeta y no en el styles.css. Tuve que configurar la extensión para que lo guardara en el lugar correcto.

Creo que esta forma de trabajar ayuda mucho para mantener el proyecto después. Si hay que cambiar el color principal se cambia una variable y listo, y si hay que arreglar la card se va directo a su archivo sin miedo de romper otra parte de la página.

En un equipo con varios programadores sirve porque todos escriben las clases de la misma forma, entonces cualquiera puede entender el código del otro. Tampoco se pisan los estilos y cada uno puede trabajar en un componente distinto al mismo tiempo. Y si todos los equipos de la agencia usan BEM, una persona puede pasar de un proyecto a otro y entender el código sin que se lo expliquen.